# INDEX

NAME [illegible] . B SUBJECT Computer networks

STD _____ DIV. _____ ROLL NO. _____ SCHOOL 24070103 dp

| SL NO | PAGE NO | TITLE | DATE | TEACHER'S SIGN / REMARKS |
|---|---|---|---|---|
| 1 | | Basic networking commands | 20/7/26 | |
| 2 | | Study of different Type of network cable | 27/7/26 | |
| 3 | | Packet Tracer Tool Installation and User Interface | 30/7/26 | |
| 4 | | Setting up a local Area network (LAN) | 4/7/26 | |
| 5 | | Experiment on Packet Capture Tool Wireshark | 27/8/26 | |
| 6 | | Hamming Code - Error detection and Correction | 1/9/26 | |
| 7 | | Sliding window | 24/9/26 | |
| 8 | | NMAP Live Hosting DIS covery DRP, ICMP, TCP/UDP | 28/9/26 | |
| 9 | | Implementation of Subnetting in cisco | 30/9/26 | |
| 10 | a | Introduction with router in cisco | 1/10/26 | |
| | b | configure of internetwork using wireless router, DHCP, Server | | |

| SR NO | PAGE NO | TITLE | DATE | ACHIEVEMENT REMARK |
|---|---|---|---|---|
| 11 | a) | Simulate STATIC Routing Configuration in Cisco | 3/10/26 | |
| | b | Simulate RIP using Cisco Packet Tracer | 5/10/26 | |
| 12 | | End-End Communication of TCP/IP layers | 8/10/26 | |
| 13 | | Implement your own Ping Program | 9/10/26 | |
| 14 | | write a code using RAW socket To implement Packet sniffing | 11/10/26 | |

Completed

23-7-2026

CS - Computer networks

Observation

Practical - 01

Basic networking Commands

Aim:

Study of various network Commands used in Linux and windows

Commands

1) arp-a is a short form of address resolution protocol which show the IP and MAC address of your routes

OUTPUT:

Interface 172.16.....0x4

| Internet address | Physical address | Type |
|---|---|---|
| 172.16.8.1 | 7C-5a-1c-cf | dynamic |
| 172.16.8.172 | 7C-5f-58-35-0d | dynamic |
| 244.00.251 | 01-00-5e-36-66 | STATIC |
| 244.00.2 | 01-00-5e-00 | STATIC |

2/ hostname

This is simple of all TCP IP Commands of the local Machine

OUTPUT:

DESKTOP-K7BNENF

3/ IP config /all:

Display detailed information about the network adapter including IP address DNS server, MAC address ETC.

OUTPUT

Windows IP Configuration

Host name . . . . DESKTOP K7BNENF

Primary DNS Suffic . . . :

node Type . . mixed

4/ nbtstat -a:

Display the netBIOS name a remote on local Computer and help Trouble Shoot BIOS

OUTPUT:

Ethernet.

node IP address . [192.168.56.1] Scop Id: [ ]

Host not found

HOTSPOT Shied network address :

node IP address : [0.0.0.0] Scode Id: [ ]

Host not found

5) netstat - an:

Display active TCP/UDP connections

listening ports, routing table and network

statics

OUTPUT.

Interface List

0x00 88 AC 54 id 06 . . HOTSPOT Shield TAP-Windows adapter V9

10.03.00 27000000a . . Virtual Box Host-only Ethernet adapter

16.0.62.f9 0c a1 99 57. Microsoft Wifi Direct Virtual adapter

6) nslookup google.com:

Queries the DNS server to obtain the IP address of a domain name

OUTPUT:

Server: LAPTOP-JPUMTAJ3

address: fe80::6369:a767:8366:1449

non-authoritative answer:

name: google.com

address: 2404:6800:4007:805::200e

142.250.67.46

7) PathPing google.com:-

Combines the feature of Ping and Tracert to trace the path to destination and calculate the packet loss at each hop

OUTPUT:

Tracing route to google.com [142.250.67.46]

Over a maximum of 30 hops:

0 DESKTOP-3GUHC7J [172.16.11.50]

1 172.16.8.1

2 137.97.136.129

3 * * *

8) Ping google.com:

Check whether another computer on website is reachable over the network using ICMP Echo request and Echo Reply Message.

OUTPUT:

Pinging google.com [142.250.67.46] with 32 Byte of data:

Reply from 142.250.67.46 : Byte = 32 Time = 2ms TTL = 118

Reply from 142.250.67.46 : Bytes = 32 Time = 2ms TTL = 118

Reply from 142.250.67.46 : Bytes = 32 Time = 7ms TTL = 118

9) route:

Display or Modifier the IP routing table used by the Operating System

OUTPUT:

Manipulates network routing table

ROUTE [-f] [-p] [-4/-6] Command [destination] [Mask netmask] [gateway] [METRIC Metric]

- f

clear the routing Table of all gateway entries. If this is used in conjuction with one of the commands, the Tables are cleared Prior.

Linux Commands.

b) ip address show:-

Display and Manages IP address routing and network interfaces

OUTPUT:

1: lo: <LOOPBACK, UP, LOWER_UP> mtu 65536

qdisc noqueue state UNKNOWN group default

link/loopback 00:00:00:00:00:00 brd

00:00:00:00:00:00

inet 127.0.0.1/8 scope host lo

valid_lft forever preferred_lft forever

11) ifconfig :-

Display network interface details such as IP address and network statistic

OUTPUT :

eth0: flags = 4163 <UP, BROADCAST, RUNNING, MULTICAST > mtu 1500

inet 17.17.197.94 netmask 255.255.240.0 broadcast 172.17.205.255

inet6 fe80::215:5dff:fe82:dc86 prefixlen 64 scopeid 0x20<link>

12) mtr google.com :

combines ping and traceroute to display the network path and performance statistic

OUTPUT :

LAPTOP-J8U1HOT4B (172.17.197.94) → google.com

| HOST | Loss % | Snt | Last |
|---|---|---|---|
| 1) LAPTOP-J8U1HOT4B.mshome.net | 0.0% | 128 | 0.7 |
| 2) 192.168.1.1 | 0.0% | 140 | 1.7 |
| 3) 100.84.0.1 | 0.0% | 163 | 5.7 |

4) (waiting for reply)

3) Tcpdump -i eth0 :-

capture and display network packets passing through a network interface for analysis

OUTPUT:

Tcpdump: Verbose OUTPUT suppressed, use -v or -vv for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes

10:16:21.123456 IP 192.168.1.10.52134 > 8.8.8.53: UDP, length 32

4) ping google.com:

sends ICMP echo request packets to verify network connectivity and measure response time

OUTPUT:

PING google.com (142.251.43.206) 56(84) bytes of data.

64 bytes from kmcca-bb-in-f14.1e100.net (142.251.43.206): icmp_seq=1 ttl=117 time=5.76 ms

Result.

Thus the various windows and linux networking commands were executed successfully



# Practical - 02

Aim: Study of different Types of network cables

a) understanding different Types of network cable

cables that are used in networking are:

1. Unshielded Twisted Pair (UTP) cable
2. shielded Twisted Pair (STP) cable
3. coaxial cable
4. fiber optic cable

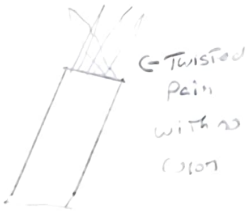
## Cable:

UTP

category 3

category 5

category 5e




advantage:

- cheaper in cost
- Easy to install

Disadvantage:

- More prone to EMI and noise

Cabel Type:

STP

STP category 6, 6a

category 7



advantages:

- Shielded
- faster than UTP

Disadvantage:

- expensive
- less susceptible to noise and interference
- requires installation effort

Cable Type:

Coaxial cable

RG-6

RG-59

RG-11

advantage:

- High Bandwidth
- Low Loss Bandwith

Disadvantage:

- limited distance
- Size is Bulky

cable

Category:

Singl Mode

Multi Mode

advantage:

- High Speed
- High Bandwidth
- High Security



Disadvantage:

- Expensive
- Requries Skilled installers

Result:

Types of network cable and Making Cross and cable through crimping tool is practiced

Ex - 3 To study the Packet Tracer tool installation and user interface overviews

Aim:

To analyse the behaviour of network device using Cisco Packet Tracer simulator

Procedure:

1) From the network component box, and drag and drop the below component

a) 4 generic PCs and 1 Hub

b) 4 generic PCs and 1 Switch

2) Click on the connections

a) Click on copper Straight Through cable

b) Select one of the PC and connect it to Hub using the cable

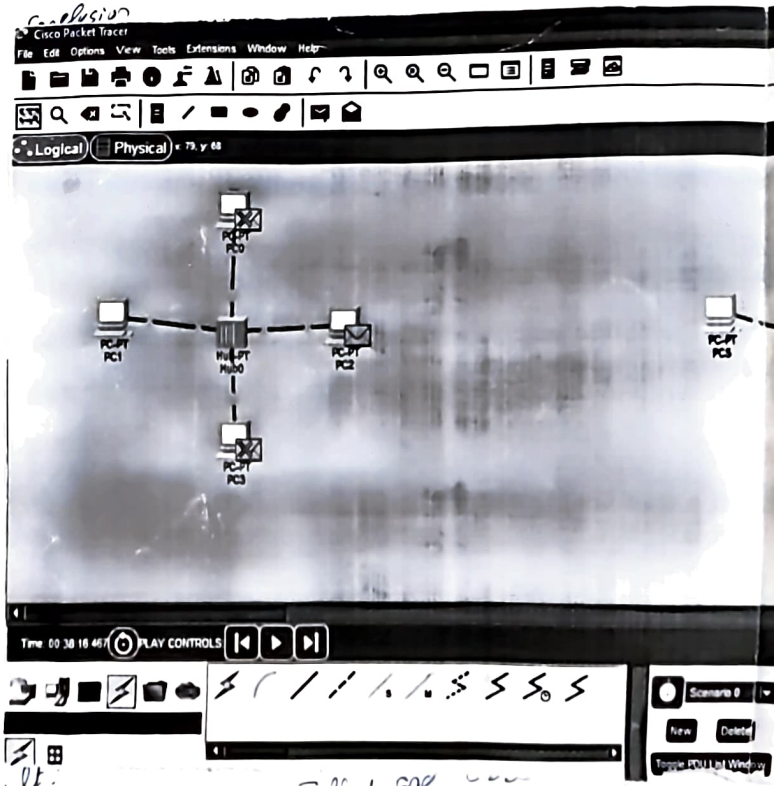
c) Similarly connect 4 PCs to the switch using copper Straight Through cable

3) Click on the PCs connected to hub go to the desktop tab, click on IP configuration. Here the default gateway and DNS server information not needed

4) Observe that flow of PDU from source PC to destination PC by selecting the realtime mode of simulation

5) Repeat steps 3 to step 5 for the PC by searching the switch

6) observe how hub and switch are forwarding the PDU and write your observation and conclusion




Result:

The Tool is installed and

Ex:04 Setting UP A LAN

Aim:

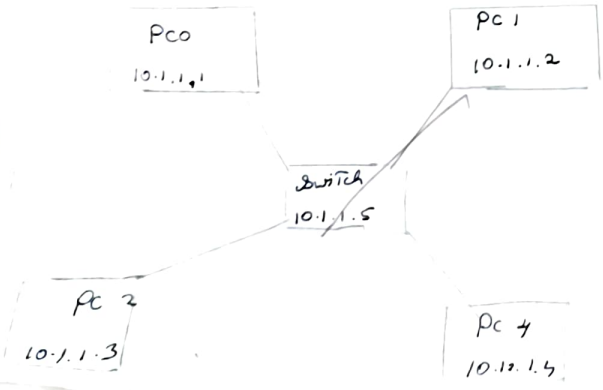
To design, SET UP, and configure a local Area network (LAN) Connecting four computers Through a network switch, assign static IP address to all device verify Connectivity using the Ping command

Equipment requirect:

| SNO | Equipment | quantity | Specification |
|---|---|---|---|
| 1 | Personal computer | 4 | Ethernet (nic) enabled |
| 2 | network switch | 1 | 8/16/24 Port |
| 3 | Ethernet cables | 4 | Straigh-Through RJ-45 Connections |

network diagram:

STAR TOPOLOGY LAN




IP address Configuration Table

| Device | IP address | Subnet mask | Connect To |
|---|---|---|---|
| Pc0 | 10.1.1.1 | 255.0.0.0 | Switch Port Fa0/1 |
| Pc1 | 10.1.1.2 | 255.0.0.0 | Switch Port Fa0/2 |
| Pc2 | 10.1.1.3 | 255.0.0.0 | Switch Port Fa0/3 |
| Pc3 | 10.1.1.4 | 255.0.0.0 | Switch Port Fa0/4 |
| Switch Managment | 10.1.1.5 | 255.0.0.0 | Switch Port Fa0/5 |

Implement in cisco Packet:

Since Physical hardware was limited The lan was also implemented and using cisco Packet also implement and unverify

STEP 1:

1. Open cisco Packet Tracer
2. Go to the device category Panel. Select end devices and drag 4 Pc
3. Select switch and drag 1 switch into The WORK SPACE

STEP 2: Cable the devices

4. Click connection → select UPPER straight Through cable

5. Click cable PC, Then click the the switch and select as available port

6. Repeat for all four PCs (Port Fa0/1 To Fa0/4) Green link light

STEP 3:- Assign ip addresses To PCs

7. Click a PC → Desktop Tab → IP configuration

8. Enter the static ip addresses & subnet Mask as ip Table

9. Click Switch → CLI Table

10. Enter The following commands To Assign Management IP

ip address 10.1.1.5 255.0.0.0

STEP 5: Verify connecting using Ping

12. using the ping command To Test connectivity To other Pc Ping 10.1.1.2

13. Successfully Reply from... * Message confirm

14. add a server device, end device → Servers Connect To switch

15. Assign an ip addressing (10.1.1.10) Subnet Mask on desktop → ip config Tab

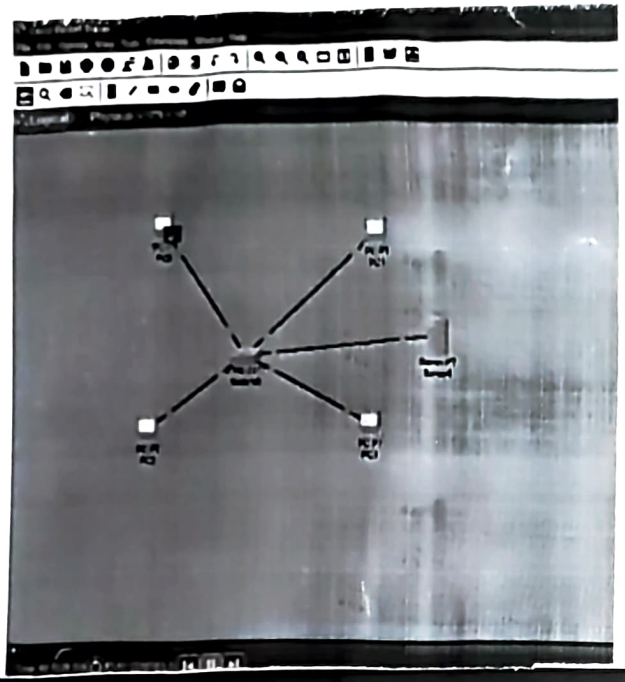
16. Open. The Server services Tab and The 17 TFTP Services

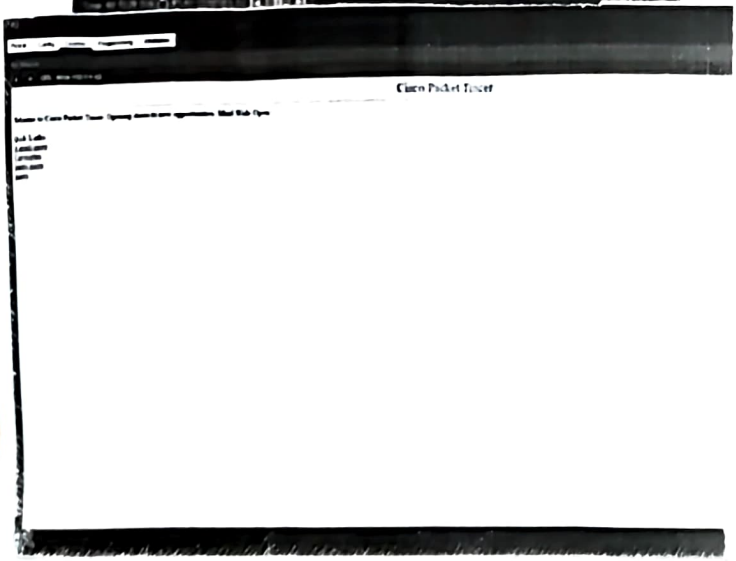
17. Switch To Simulation mode

Observation:

All 4 Pcs were successfully connected To The switch in a Star Topology & unique static ip address within The same subnet

Ping Tests b/w all devices






Result:

The Lan Pcb (experiment was successfully built, configured, and verified for ip Connectivity and client server

EXP NO:5 Experiment on capturing Tools

WIRE SHARK

Aim:

To understand the features of wireshark as a packet capture tool and analyze protocol encapsulation at various layers of a protocol stack

Tools Required:

Pc connected to Lan, wireshark software

Theory:

wireshark is a passive network packet analyze that capture network frames in real time

- packet list
- Packet details
- Packet Bytes

Procedure:

1. general capture:

Open wireshark, Select the active interface set capture client to 100

2 TCP/UDP filter:

Type TCP (or) UDP in the display filter bar

3 ARP filters:

Types arp in the display filter

clear to view address resolution request and

replies

4. DNS filter:

Types dns in the display filter this

To observe domain name and query

5. HTTP filter:

Types HTTP in display filter be to

ins Pcct

6. IP / ICMP filter:

Type icmp (or) ip in the

display filter leal To view Ping request/

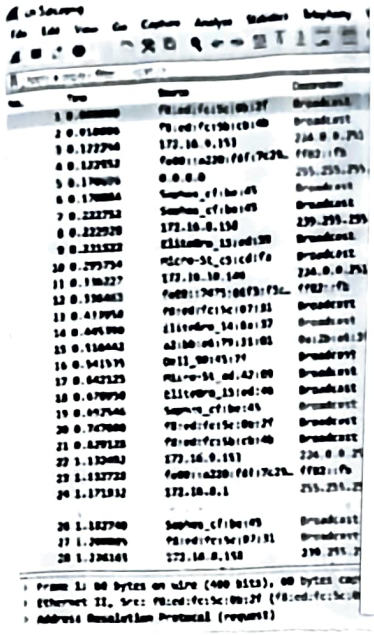
reply frames

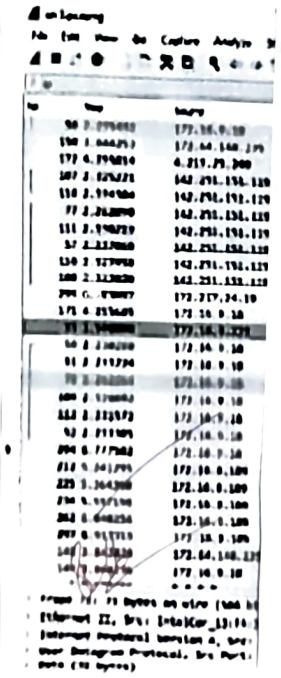
7. DHCP filter:

Types DhcP (or) bootP in

The Display filter clear To analyze IP

Lease Packets





Exp no: 06

Date: 10.9.20

# Hamming code - error detection and correction

Aim:

To write hamming code for error detection and correction

Code:

```
def Hamming - Encryption (data):
    k = len (data)
    l = 0
    while (2** l) <= (k + l + 1):
        r+ = 1
        r - = 1
        n = k + r + 1
    code = [0] * n
    j = 0
    for i in range (1, n):
        if (i & (i-1) != 0):
            code [i] = data [j]
            j+ = 1
    for j in range (1, n)
        if 2 & Pos:
            Pair = int [code [j]]
    code [Pos] - str (Pair)
```

```
def hamming code - decryption (data, k);
    for i in range (r);
        Pos = 2+i;
        Pair = 0
        for j in range (1, k+k+1):
            if j < Pos;
                Pair += int (code [j])
        if (Pair != 0)
            if [data [Pos] == 0):
                data [Pos] = '1'
            else:
                data [Pos] = '0'
    return '' join (data)
    data = input ("Enter Binary data:
    Print ("Data after encoding from
        Sender side:" hamming
        encryption)
    rdata = input ("enter data received
        by the receive")
    Print (hamming code - Decryption
    (or data, or, Rdata length))
```

O/P:

Enter Binary Data = 1011

Data after encoding from

Sender Side = 01100110

Enter Data received by

01101111

Data after decoding 01100111

Result:

Thus the Hamming code was dicted and Corrected the errors successfully

Exp. 07 Sliding Window Protocol

Date 24.9.26

Aim:

To implement flow control at data link layer using sliding window protocol the flow from one node to another node

Sender Program:

1. INPUT window size from the user
2. INPUT a text Message from the user
3. consider character per frame
4. Create a frame with following fields

   [ frame no, DATA ]
5. Send the frames
6. wait for the acknowledgement from the Reciver
7. Reciver a file called Reciver buffer
8. Check ACK filled for the acknowledgement

Create a receive file with following

1. Reader a field called Sender - Buffer
2. check the frame no
3. If the frame no. are as expected

write the appropriate Ack in the Reciver Buffer

Code:

```
import Time
import os
Send_file = "Sender_Buffer.txt"
Recv_file = "Reciver_Buffer.txt"
message = input ("Enter msg")
window_size = int (input ("Enter window size"))

frames[]
for i, ch, in enumerate (message);
    frames.append ((i+1, ch))

start=0
while start < len (frames):
    end = min (start + window_size)
              len (frames))
with open (Sender_file, "w") as f:
    for i in range (start, end);
    Print (f"frame no: {frames[i][0]}
           DATA : {frames[i][1]}
    Print ("Waiting for acknowledgment...")
    Time.sleep (2)
    if not os.path.exist (receiver_file);
        Print ("Reciever Buffer not found.")
        break
    with open (receiver_file, "r") as f:
        ack = f.read().strip()
    Print ("Recived Response:", ack)
    if ack == end;
        Print ("Acknowledgement received:")
        start = end
    else Print ("unexpected Ack")
elif ack start ("NACK"):
    Nack_no = int (ack split (" ") [1])
    Print (f"NACK received for frame")
    start = nack_no - 1
Print ("In All frames Transmitted")
```

Reciver Program

```
with open ("Sender_Buffer.txt") as f
    frames = f.readlines()
end Packet =
```

```
for frames in frames:
    no, data = frame.strip().split(":")
    if int(no) == expected:
        print("frame", no)
        expected += 1
    else:
        print("NACK", expected)
        break
else:
    with open("Recive Buffer.txt", "w")
    f.write("Ack:" (expected - 1))
    print("Ack", expected - 1)
```

INPUT:

Enter the Total Message : Hello

Enter Window Size : 3

OUTPUT (Sender):

Frame no: 1 DATA: H

Frame no: 2 DATA: E

Frame no: 3 DATA: L

Waiting for Acknowledgment

OUTPUT - Recive

Recived Frame no: 1 DATA: H

Frame accepted

Recived Frame no: 3 DATA E

Frame accepted

Recived Frame no: 3 DATA: L

Frame accepted

Result:

Thus Sliding window is implement Successfully

Experiment: 8

28/9/24

ARP, ICMP and ICP/UDP

Aim:

To discover live hosts on a network using NMAP host discovery scans and To understand which protocol operate at which layer of TCP/IP

ARP Host Discovery

$sudo nmap -PR -sn 10.200.6.0/24

O/P:

Starting Nmap 7.8 (https://nmap.org)

Nmap scan report for 10.200.6.1

Host is up

Scanned in 10.51 seconds

ICMP Host discovery

1) ICMP Echo (-PE)

$ sudo nmap -PE -sn 10.200.6.0/24

Nmap scan Report for 10.200.6.1

HOST is UP (0.0691 latency)

Nmap done: 256 IP address

Scanned is 8.44 second

ICMP Timestamp (-PP)

$ nmap -PP is 1.0 200-6.0/24

Nmap Scan Report for 10.200.6.53

Host is UP (0.0605 latency)

Nmap Scan Report for 10.200.6.253

HOST is UP (0.069 latency)

ICMP Address Mask (-PM)

$ nmap -PM -sn 10.200.6.1/24

Nmap done: 256 IP address Scanned in 52.14 second

TCP & UDP Ping Discovery

i) TCP SYN Ping

$ nmap -B -sn 10.200.6.1/24

Nmap Scan report

Host is UP (0.70 latency)

Nmap Scan Report for 10.200.6.82

ii) TCP Ack Ping (-PA)

$sudo nmap -PA -sn 10.200.6.0/24

Nmap Scan report for 10.200.6.153

HOST is UP (0.0705 latency)

Host is UP

iii) UDP Ping (-PU)

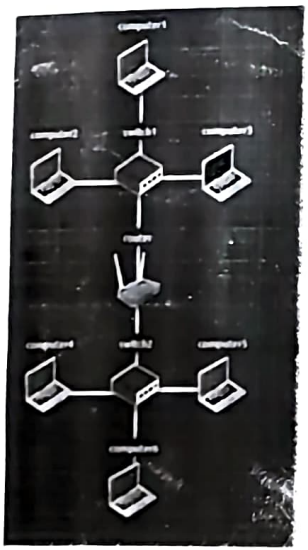
$ sudo nmap (-PU -sn 10.200.6.0124

Nmap scan report for 10.200.6.52

Host is up (0.205 latency)

Nmap scan report for 10.200.6.250

Host is up (0.693 latency)





Result:

Thus the host discovery for ARP, ICMP, TCP, UDP was studied and the O/P verified

EXP 10.04

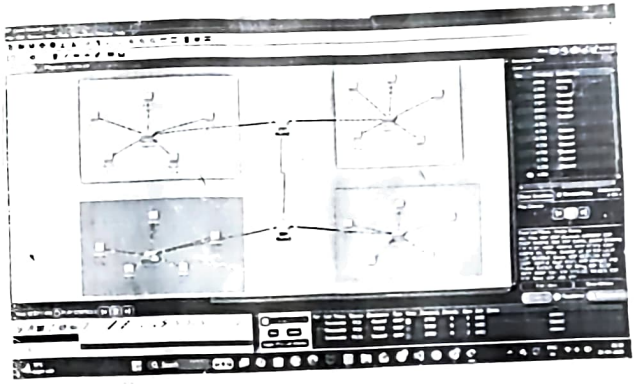
DATE 28.9.26

# Implementation of Cisco Packet Tracer

**Aim:** To implement the subnetting in Cisco Packet Tracer simulation

Procedure:

- Open Cisco Packet Tracer and create a new network topology
- add the router switches and PC to workspace
- connect the device using copper cables
- use the network 192.168.10.214 and divide it into 127 network
- configure the routers
  - use exit and configure terminal in the using IP using
    - i) ip address
    - ii) no shutdown
- configure the required switch port in access mode
- assign each PC its IP address subnet mask and gateway
- use the ping command to test
- if ping is successful network is configured correctly



Result:

Thus, Implementation of Subnetting in Cisco Packet, is done successfully

EXP NO 10 Introducing with packet tracer CISCO

Packet Tracer SIMULATION

EXP NO 10(b) Design and configure an interNetwork using wireless routers, DHCP, Basic and Internet cloud

Aim:

To design and configure a simple internetwork using a router in Cisco packet Tracer and test connectivity

Procedure:

STEP 1. Configure router

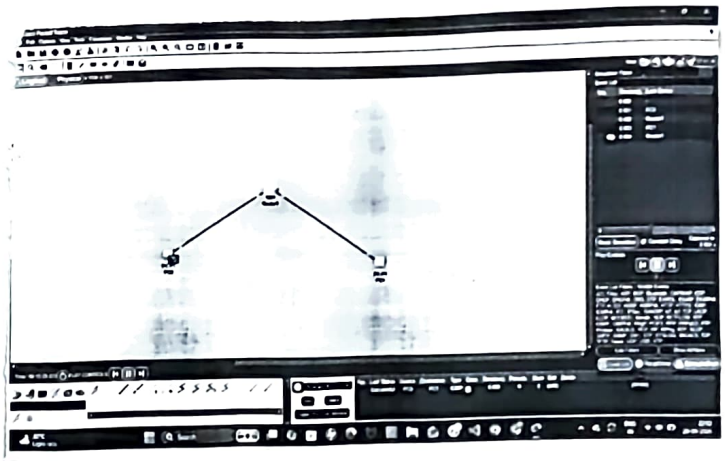
- Select the ROUTER
- Press Enter to START
- Type enable → To activate the Privileged Mode

STEP 2:

- Assign IP address To every PC in network
- Select the PC go To DESKTOP and select IP configuration and assign IP address

STEP 3:

- Connect fast Ethernet, of Port of PC0 with fast Ethernet



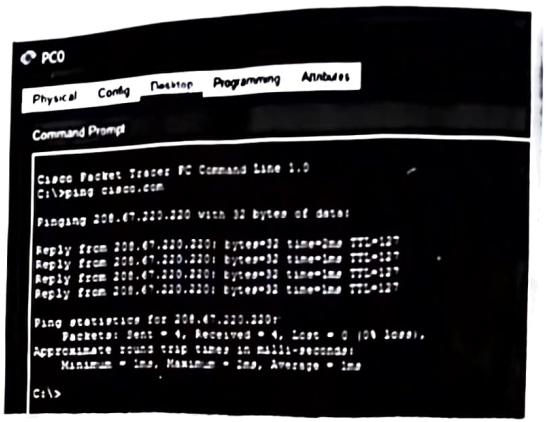
Ex.No 12(a) Design and Configure an Internetwork using wireless router, DHCP Service and internet cloud

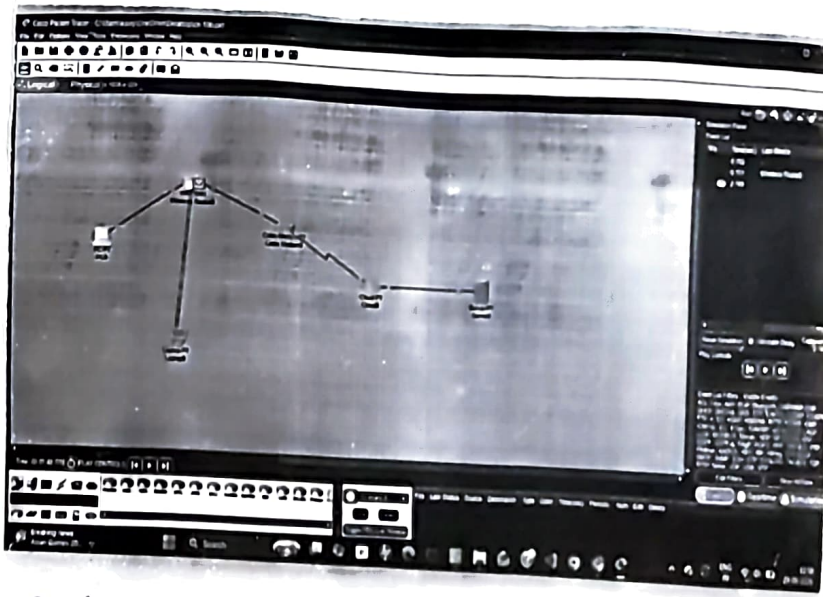
Aim:

To design and configure an internetwork using a Wireless Router DHCP and internet cloud in Cisco Packet Tracer

Procedure:

- Open Packet Tracer and Create the required network topology
- Connect PC, Laptop, To router Internet cloud Server
- Connect the wireless router with DCP Home Network
- Configure the Server with the IP 208.67.220.220 and Enable DHCP and DNS
- Verify Connectivity using



Result:

The internetwork was successfully designed and configured using a router DHCP and internet cloud and the connectivity was successfully

Exp 11a Simulate static routing configuration using Cisco Packet TRACER

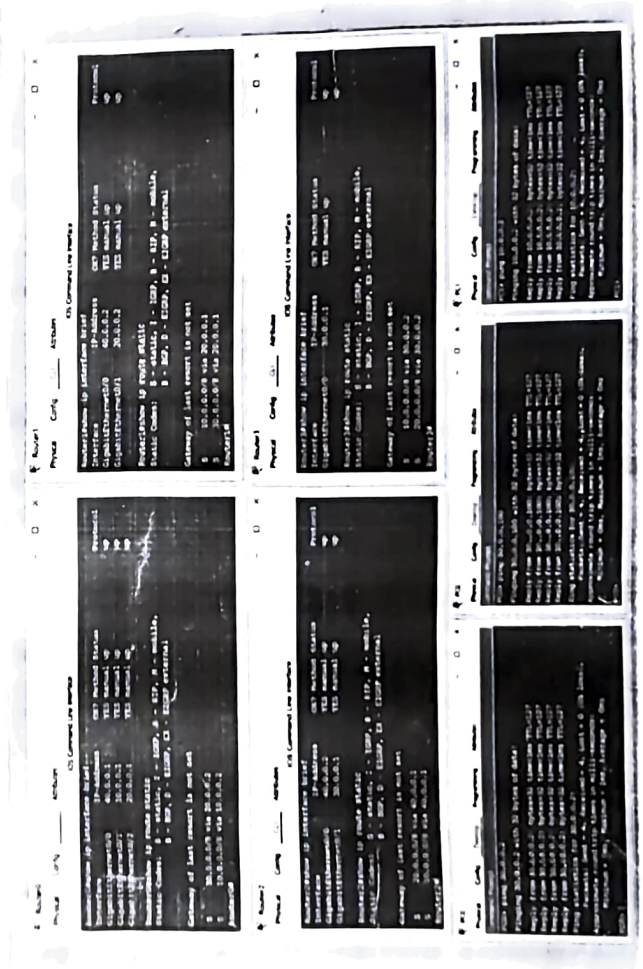
Aim:
To simulate a static routing configuration using Cisco Packet

Procedure:

- Open Cisco Packet Tracer and To give network topology
- Connect 3 routers and using suitable Table
- Configure IP address for routers and PC
- Enable all router interface using router and PC
- Configure static routing Router 0, and 1
- Set Main and backup routers using different administrative distance
- Test Connectivity
- Use Trace To verify The routing path

Exp 11b:

Aim:

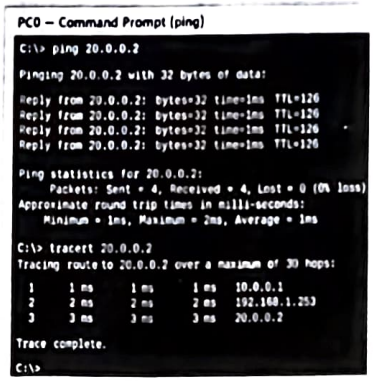


Exp # 6

Aim: To simulate RIP using cisco Packet Tracer

Procedure:

- Create the Topology with 3 routers and 2PC
- Connect the device using suitable cables
- Assign the given IP address to both
- Enable interface using no shut down
- Test connectivity using ping 20.0.2
- Use Tracert 20.0.0.2 To verify the routes
- Disconnect the direct routers & Router click and verify




Result:

Thus the simulation of RIP using cisco Packet Tracer is verified successfully

Exp 12 End - End Communication at Transport Layer

a) Implement echo client server using TCP/UDP sockets

b) Implement a chat Program using socket Program

TCP:

```
Import socket
Import Threading

def start_server:
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('127.0.1.1', 5000))
    server.listen()
    Print(f"SERVER] Connected By {addr}")
while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit'
            Print("SERVER] Connection closed By client.")
            break
        Print(f"SERVER] Recived : {data}")
        conn.send(f"Server Recived : {data}".encode())
    conn.close()
    server.close()
```

# Output:

```
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 53458)Enter message (or 'quit' to exit):
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
Enter message (or 'quit' to exit):
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 60328)Enter message (or 'quit' to exit):
HI
[SERVER] Received: HI
[SERVER RESPONSE] Server received: HI
Enter message (or 'quit' to exit): quit
[SERVER] Connection closed by client.
```

```
def start_client():
    client = socket.socket(socket.AF_INET, socket)
    client.connect(('127.0.0.1', 5000))
    print("[Client] Connected to server")

    while True:
        msg = input("Enter Message (or 'quit' to exit):")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"\n [server response*]

    client.close()

if __name__ == "__main__":
    server_thread = threading.Thread(target =
    target)
        server_thread.start()

    start_client()
```

Enter message (or 'quit' to stop): [SERVER] Running on 127.0.0.1:12000hi

[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from Server: hi
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 49670): hello
Echo from Server: hello
Enter message (or 'quit' to stop): how are you
[SERVER] Received from ('127.0.0.1', 49670): how are you
Echo from Server: how are you
Enter message (or 'quit' to stop): quit
[CLIENT] Closing connection...[SERVER] Client ('127.0.0.1', 49670) ended the connection.

[CLIENT] Stopped.[SERVER] Stopped.

UDP

```
import socket
import threading

def udp_server():
    s = socket.socket(socket.AF_INET,
    socket.SOCK_DGRAM)
    data, addr = s.recvfrom(1024)
    msg = data.decode()
    while msg.lower() != 'quit':
        print("[Server] Received from client:", msg)
        s.sendto(data, addr)
        data, addr = s.recvfrom(1024)
        msg = data.decode()
    print("[Server] Client ended the connection.")
    s.close()

def udp_client():
    c = socket.socket(socket.AF_INET,
    socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or 'quit' to stop): ")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            break
        data, _ = c.recvfrom(1024)
        print("Echo from Server:", data.decode())
    print("[Client] Closing connection...")
    c.close()
```

13. implement your own Ping program

```
import os
import platform
import subprocess

def ping(host = "google.com", count = 4):
    param = "-n" if platform.system().lower()
== "windows" else "-c" command = ["ping",
param, str(count), host]

try:
    output = subprocess.check_output(command,
universal_newlines = True) print(output
except exception as e:
    print(f"Ping failed: {e}")

if __name__ == "__main__":
    ping("8.8.8.8")
```

OUTPUT:

14

Write a code using RAW Sockets to implement Packet sniffing

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP

def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst

    # Determine the protocol
        protocol_name = ""
        if protocol == 1:
            protocol_name = "ICMP"
        elif protocol == 6:
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "Unknown Protocol"

    # Print packet details
        print(f"Protocol: {protocol_name}")
        print(f"Source IP: {src_ip}")
        print(f"Destination IP: {dst_ip}")
        print("-" * 50)
```